

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Deploy Spring Boot Applications on External Tomcat Server

Table of Contents

Learning Outcome:	3
1. Introduction	4
2. Configuring Spring Boot Application for Deployment	5
Adding necessary dependencies for Servlet Initializer in the project	6
Configuring the Servlet Initializer in the application	7
3. Setting the Main Class	8
Configuring the main class for deployment in the Tomcat server	8
4. Updating Packaging from JAR to WAR	9
Modifying the project configuration to generate a WAR file instead of a JAR file	10
Handling dependencies and configuration changes for a WAR deployment	11
5. Packaging the Application	12
Verifying the generated WAR file structure	13
Checking for any packaging issues or conflicts	13
6. Deployment in Tomcat	14
Deploying the packaged WAR file on the Tomcat server	14
Verifying the deployment and accessing the Spring Boot application	15

UNIT 9: Deploy Spring Boot Applications on External Tomcat Server

Objectives:

In this Unit you will learn –

- Understand the concept of deploying Spring Boot applications on an external Tomcat server.
- Configure a Spring Boot application to run as a WAR file.
- Utilize the Spring Boot Servlet Initializer to configure the application for deployment.
- Set the main class for the Spring Boot application.
- Update the packaging from JAR to WAR.
- Package the Spring Boot application.
- Deploy the packaged application on a Tomcat server.

Learning Outcome:

Deploy Spring Boot Applications on Tomcat Server

9. Deploy Spring Boot Applications on External Tomcat Server

1. Introduction

When developing Spring Boot applications, one common choice for deployment is to use an external Tomcat server. Tomcat is a popular Java-based web server and servlet container that provides an environment for running Java web applications. By deploying a Spring Boot application on Tomcat, you can leverage the capabilities of Tomcat for managing and serving your application.

Benefits of deploying Spring Boot applications as WAR files:

- **Compatibility:** Tomcat supports the deployment of web applications in the form of Web ARchive (WAR) files. By packaging your Spring Boot application as a WAR file, you can ensure compatibility with Tomcat and other servlet containers.
- **Standardization:** Deploying as a WAR file follows the standard Java EE deployment model, making it easier to deploy and manage your application across different environments and servers.
- **Servlet Container Features:** By deploying on Tomcat, you can take advantage of the various features and capabilities provided by the servlet container, such as session management, request handling, security configurations, and more.
- **Scalability:** Tomcat supports clustering and load balancing, allowing you to scale your Spring Boot application to handle increased traffic and load.
- **Integration with Existing Infrastructure:** Deploying on Tomcat allows for integration with other tools and systems commonly used in enterprise environments, such as monitoring tools, load balancers, and web servers.

Considerations when deploying Spring Boot applications as WAR files:

- **Application Structure:** Deploying as a WAR file requires organizing your Spring Boot application into the standard Java EE structure, including the necessary web-related files such as web.xml, servlets, and filters.

- **Servlet Container Dependencies:** Depending on the features and dependencies your application requires, you might need to include additional dependencies related to the servlet container in your project.
- **Configuration Changes:** When deploying as a WAR file, certain configuration changes might be required to ensure compatibility with Tomcat. For example, you might need to update the server-specific configuration files or adjust the application's context path.
- **Deployment Process:** The deployment process for a WAR file involves packaging the application, deploying it on the Tomcat server, and managing the necessary configurations. Understanding this process and its intricacies is crucial for successful deployment.

By understanding the benefits and considerations of deploying Spring Boot applications as WAR files on Tomcat, you can make informed decisions and effectively leverage the capabilities provided by the servlet container for your application deployment.

2. Configuring Spring Boot Application for Deployment

The Spring Boot Servlet Initializer plays a crucial role in configuring a Spring Boot application for deployment in a servlet container like Tomcat. It allows Spring Boot to adapt to the servlet container's deployment structure and initialize the application properly.

When a Spring Boot application is packaged as a JAR file, it includes an embedded servlet container (such as Tomcat, Jetty, or Undertow). This embedded container starts the application automatically. However, when deploying as a WAR file in an external servlet container like Tomcat, the application needs to be configured differently.

The Spring Boot Servlet Initializer is responsible for initializing the Spring application context and registering the necessary servlets, filters, and listeners required for the application to run correctly in the servlet container.

Adding necessary dependencies for Servlet Initializer in the project

To enable the Spring Boot Servlet Initializer and configure the application for deployment in a servlet container, you need to include the necessary dependencies in your project.

1. Open your project's pom.xml file (for Maven) or build.gradle file (for Gradle).
2. Add the following dependencies to your project:

For Maven:

```
<dependencies>

    <!-- Other dependencies -->

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-web</artifactId>

    </dependency>

    <dependency>

        <groupId>javax.servlet</groupId>

        <artifactId>javax.servlet-api</artifactId>

        <scope>provided</scope>

    </dependency>

</dependencies>
```

For Gradle:

```
dependencies {

    // Other dependencies

    implementation 'org.springframework.boot:spring-boot-starter-web'

    providedCompile 'javax.servlet:javax.servlet-api'

}
```

3. Save the changes and let your build tool resolve the dependencies.

Configuring the Servlet Initializer in the application

Once you have added the necessary dependencies, you need to configure the Servlet Initializer in your Spring Boot application.

1. Create a new class in your project, typically named `ServletInitializer`, and make it extend `SpringBootServletInitializer`.
2. Override the `configure()` method in the `ServletInitializer` class.
3. Within the `configure()` method, invoke the `configure()` method of the `SpringApplicationBuilder` class and pass your main application class as an argument.

```
import org.springframework.boot.builder.SpringApplicationBuilder;

import
org.springframework.boot.web.servlet.support.SpringBootServletInitializer;

public class ServletInitializer extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder
application) {

        return application.sources(YourMainClass.class);
    }
}
```

4. Replace `YourMainClass` with the actual main class of your Spring Boot application.

3. Setting the Main Class

The main class of a Spring Boot application is the class that contains the main method, which serves as the entry point for the application. This class is responsible for bootstrapping the Spring Boot application and starting the embedded servlet container when running the application as a standalone JAR.

Typically, the main class is annotated with `@SpringBootApplication` or one of its composed annotations, such as `@EnableAutoConfiguration`, `@ComponentScan`, and `@Configuration`. The `@SpringBootApplication` annotation enables various auto-configuration features and component scanning within the application.

Configuring the main class for deployment in the Tomcat server

When deploying a Spring Boot application on a Tomcat server, you need to configure the main class so that Tomcat knows which class to use as the entry point for the application.

1. Open the `pom.xml` file (for Maven) or `build.gradle` file (for Gradle) of your Spring Boot project.
2. Ensure that the `spring-boot-starter-web` dependency is included in your project's dependencies. This dependency provides the necessary classes and configurations for running a Spring Boot application as a web application.
3. Locate the `<packaging>` element in your `pom.xml` file or the `apply plugin` section in your `build.gradle` file. By default, Spring Boot applications are packaged as JAR files. To deploy on Tomcat, you need to update the packaging to WAR.

For Maven:

```
<packaging>war</packaging>
```

For Gradle:

```
apply plugin: 'war'
```

4. Save the changes to the `pom.xml` or `build.gradle` file.

5. Open your main class (the one with the main method).
6. Add the extends `SpringBootServletInitializer` to the class declaration to inherit from the `SpringBootServletInitializer` class.

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
    org.springframework.boot.web.servlet.support.SpringBootServletInitializer;

@SpringBootApplication

public class YourMainClass extends SpringBootServletInitializer {

    public static void main(String[] args) {

        SpringApplication.run(YourMainClass.class, args);

    }

}
```

7. Replace `YourMainClass` with the actual name of your main class.

4. Updating Packaging from JAR to WAR

When developing Spring Boot applications, you have the option to package your application as either a JAR (Java ARchive) or a WAR (Web ARchive) file. Understanding the differences between these packaging formats is essential when deploying your application on a servlet container like Tomcat.

JAR (Java ARchive):

JAR files are typically used for standalone Java applications. They contain all the

compiled classes, resources, and dependencies required to run the application. JAR files are executed using the `java -jar` command and include an embedded servlet container when using Spring Boot. JAR packaging is suitable for running Spring Boot applications independently without the need for an external servlet container.

WAR (Web ARchive):

WAR files are used for web applications that run within a servlet container. They include web-related components such as servlets, JSP files, HTML, CSS, and other static resources. WAR files also contain the necessary Java classes, resources, and dependencies specific to the web application. WAR packaging follows the standard Java EE deployment model and is suitable for deploying Spring Boot applications on servlet containers like Tomcat.

Modifying the project configuration to generate a WAR file instead of a JAR file

To update the packaging of your Spring Boot application from JAR to WAR, you need to modify the project configuration files.

1. Open your `pom.xml` file (for Maven) or `build.gradle` file (for Gradle).
2. Update the `<packaging>` element in your `pom.xml` file or the `apply plugin` section in your `build.gradle` file to change the packaging to WAR.

For Maven:

```
<packaging>war</packaging>
```

For Gradle:

```
apply plugin: 'war'
```

3. Ensure that your project dependencies and configurations are compatible with the

WAR packaging.

- Review your dependencies and make sure they are suitable for a web application. Include the necessary dependencies for servlets, JSPs, and any other web-related components your application requires.
- Check your project's configuration files (e.g., `application.properties`, `application.yml`) for any settings specific to a JAR packaging. Update or remove any configurations that are not applicable to a WAR deployment.

Handling dependencies and configuration changes for a WAR deployment

When switching from JAR to WAR packaging, you might need to handle dependencies and configuration changes specific to a WAR deployment.

Dependencies:

- Ensure that your project includes the necessary dependencies for web applications, such as `servlet-api`, JSP dependencies, and any other libraries required for web-related functionality.
- Check if any of your existing dependencies conflict with the servlet container's provided dependencies. Exclude or replace conflicting dependencies if necessary.

Configuration:

- Review your application's configuration files and make any necessary changes specific to a web application. For example, you might need to modify database connection settings, session management configurations, or security configurations.
- Update any paths or URLs that might have changed due to the new deployment structure.

Web-related resources:

- Move your web-related files, such as HTML, CSS, JavaScript, JSP, and other static resources, into the appropriate directories within the project structure.
- Ensure that your servlets, filters, and other web components are correctly configured.

By modifying the project configuration, handling dependencies, and making necessary configuration changes, you can successfully update the packaging of your Spring Boot application from JAR to WAR. This enables you to deploy the application as a web application on a servlet container like Tomcat.

5. Packaging the Application

To package your Spring Boot application as a WAR file, you need to build the project using your build tool (Maven or Gradle). This will generate the WAR file that can be deployed to a servlet container like Tomcat.

For Maven:

1. Open a terminal or command prompt.
2. Navigate to the root directory of your Spring Boot project, where the pom.xml file is located.
3. Run the following command to build the project and generate the WAR file:\

```
mvn clean package
```

This command will compile your code, run tests, and package the application into a WAR file. The generated WAR file will be located in the target directory.

For Gradle:

1. Open a terminal or command prompt.

2. Navigate to the root directory of your Spring Boot project, where the build.gradle file is located.
3. Run the following command to build the project and generate the WAR file:

```
gradlew clean build
```

This command will compile your code, run tests, and package the application into a WAR file. The generated WAR file will be located in the build/libs directory.

Verifying the generated WAR file structure

After successfully building the project as a WAR file, you can verify the generated WAR file's structure to ensure it contains the required files and directories.

1. Locate the generated WAR file (e.g., your-application.war).
2. Rename the file extension from .war to .zip.
3. Extract the contents of the ZIP file to a directory of your choice.
4. Review the extracted contents and check for the following items:
 - WEB-INF directory: This directory should contain the web.xml file and any other configuration files specific to your application.
 - Classes directory: This directory should contain the compiled Java classes of your application.
 - Lib directory: This directory should contain the dependency JAR files required by your application.

Checking for any packaging issues or conflicts

When packaging the application as a WAR file, it's important to check for any packaging issues or conflicts that may arise.

1. Ensure that there are no duplicate or conflicting dependencies included in the generated WAR file. Conflicting dependencies may cause runtime issues or unexpected behavior.

2. Check for any warnings or errors during the build process that may indicate packaging issues. Review the build logs for any relevant messages.
3. Verify that all required resources, such as static files, templates, and configuration files, are properly included in the generated WAR file.
4. Test the generated WAR file by deploying it to a servlet container like Tomcat and verify that the application functions as expected.

6. Deployment in Tomcat

Apache Tomcat is a popular open-source Java servlet container that implements the Java Servlet, JavaServer Pages (JSP), and WebSocket technologies. It provides a runtime environment for deploying and running web applications.

Tomcat follows a specific deployment structure where each web application is deployed as a separate directory within the Tomcat installation. The directory structure typically includes the following components:

- **conf:** Contains the configuration files for Tomcat, such as `server.xml` and `web.xml`.
- **webapps:** The base directory for deploying web applications. Each web application is represented by a separate directory inside `webapps`.
- **work:** Stores temporary files generated by Tomcat, such as compiled JSPs.

Deploying the packaged WAR file on the Tomcat server

To deploy your Spring Boot application on the Tomcat server, you need to copy the packaged WAR file to the appropriate directory within the Tomcat installation.

1. Stop the Tomcat server if it is already running.
2. Navigate to the `webapps` directory in your Tomcat installation.
3. Copy the generated WAR file (`your-application.war`) from the build output

directory (target for Maven or build/libs for Gradle) and paste it into the webapps directory.

4. Start the Tomcat server.

Tomcat will automatically extract the contents of the WAR file and deploy the application. It will create a directory with the same name as the WAR file (e.g., your-application) within the webapps directory.

Verifying the deployment and accessing the Spring Boot application

After deploying the WAR file, you can verify the deployment and access your Spring Boot application.

1. Open a web browser.
2. Enter the following URL in the address bar:

```
http://localhost:8080/your-application/
```

3. Replace your-application with the name of the deployed application directory.
If you have modified the default Tomcat port, replace 8080 with the appropriate port number.
4. If the deployment is successful, you should see your Spring Boot application's homepage or landing page.